Software Design Patterns Research Output

Course: IT342 – System Integration

Project: Disaster Aid Connect

Submitted by : Godwin M. Labaya

Date: 04/07/2026

I. Creational Patterns

1. Singleton Pattern

Category: Creational

Problem it solves: Ensures that a class or module has only one instance throughout the application's lifetime, preventing duplicate connections or conflicting states.

How it works: The pattern restricts instantiation of a class to a single object. Any part of the app that requests the instance gets the same one, not a new copy.

Real-world example: A database connection pool in a backend server, you don't want 50 different connections being created every time a user makes a request. One shared connection is managed and reused.

Use case in DisasterAidConnect: Your supabaseClient.js file is a textbook Singleton. You create the Supabase client once and export it, then every file, Login.js, Map.js, Requests.js, Dashboard.js, imports and reuses that same instance. If each component created its own Supabase client, you'd have connection overhead and potentially inconsistent auth states.

2. Factory Pattern

Category: Creational

Problem it solves: Centralizes object creation logic so that the caller doesn't need to know the exact class or configuration details of what it's creating.

How it works: A factory function or method takes input parameters and returns the appropriate object. The creation logic lives in one place rather than being scattered throughout the codebase.

Real-world example: In a notification system, a factory might take a type like "email", "sms", or "push" and return the correct notification handler object without the caller needing to know the implementation details.

Use case in DisasterAidConnect: The makeIcon() function in Map.js is a factory. It takes a color and an isTemp boolean, then constructs and returns the appropriate Leaflet divIcon object. Callers just pass in parameters, they don't build the SVG or configure the icon anchor themselves. You could extend this factory to support more icon types (e.g., verified incidents, government-reported ones) without changing any marker rendering code.

II. Structural Patterns

3. Facade Pattern

Category: Structural

Problem it solves: Provides a simplified interface to a complex subsystem, hiding the internal complexity from the rest of the application.

How it works: A facade class or module wraps multiple complex operations behind a clean, easy-to-call interface. The client interacts only with the facade, not the underlying system.

Real-world example: A payment processing facade might expose a single processPayment(amount, method) function, while internally handling fraud checks, currency conversion, bank API calls, and receipt generation.

Use case in DisasterAidConnect: Your Supabase calls across the app act as a facade over the underlying PostgreSQL database, authentication system, and real-time subscriptions. When you call supabase.from("disasters").select("*"), you're not writing raw SQL or managing HTTP requests, supabase's client library facades all of that complexity. You could formalize this further by creating a disasterService.js file in your services/ folder that wraps all disaster-related Supabase calls, giving the rest of your app a clean interface like disasterService.getAll() or disasterService.delete(id).

4. Composite Pattern

Category: Structural

Problem it solves: Lets you treat individual objects and compositions of objects uniformly, making it easier to build tree-like or nested UI structures.

How it works: Components are arranged in a tree where both individual items (leaves) and groups (composites) implement the same interface. The parent doesn't need to distinguish between a single child or a group.

Real-world example: A file system where both a single file and a folder (which contains files) share operations like open(), delete(), or getSize(). The same operation works recursively without special cases.

Use case in DisasterAidConnect: Your React component tree follows the Composite pattern. The MapPage component is a composite that contains Sidebar, DisasterForm, ConfirmDialog, FlyTo, ClickHandler, and MapContainer, each of which may itself contain

more components. React's model inherently uses this pattern: every component, whether a leaf like a <button> or a complex container like RequestsPage, is rendered and managed the same way.

III. Behavioral Patterns

5. Observer Pattern

Category: Behavioral

Problem it solves: Defines a one-to-many dependency so that when one object changes state, all its dependents are automatically notified and updated, without tight coupling between them.

How it works: A subject (publisher) maintains a list of observers (subscribers). When the subject's state changes, it broadcasts the update to all registered observers, who then react accordingly.

Real-world example: Social media notifications when someone posts, all their followers (observers) receive a notification automatically. The poster doesn't manually contact each follower.

Use case in DisasterAidConnect: React's useState and useEffect hooks implement the Observer pattern under the hood. For example, in Map.js, when disasters state changes (a new one is added), all components observing that state the marker list on the map, the left panel list, the selected card automatically re-render with updated data. Supabase also supports real-time subscriptions (supabase.channel()), which you could use to let all connected users see new disaster pins appear on the map the moment someone else adds one, a direct application of the Observer pattern.

6. Strategy Pattern

Category: Behavioral

Problem it solves: Defines a family of interchangeable algorithms or behaviors and lets you switch between them at runtime without changing the code that uses them.

How it works: Each behavior is encapsulated in its own class or function. A context object holds a reference to one strategy and delegates the work to it. The strategy can be swapped out dynamically.

Real-world example: A sorting feature in an e-commerce app where products can be sorted by price, rating, or relevance. Each sorting method is a separate strategy, and the user's choice at runtime determines which one is applied.

Use case in DisasterAidConnect: The filtering logic in Requests.js demonstrates the Strategy pattern informally. You have multiple filtering strategies filter by status tab, filter by search keyword, filter by date and they are applied together dynamically based on user input. This could be formalized further: each filter type becomes its own strategy function, and a filter engine composes them. Additionally, the getSeverityColor() and getStatusStyle() helper functions in Requests.js are interchangeable strategies for computing display properties based on a disaster's current state swap the input, get a different visual output, without touching the rendering code.